
ENTREGA FINAL · AI AUTOMATION · CODERHOUSE

Ecosistema de Automatización IA *Gestión de Frees*

Sistema autónomo de intake, evaluación y respuesta de solicitudes de entradas de cortesía para tres marcas de eventos. Orquestación en Make, memoria relacional en Airtable, razonamiento con Claude y salida por Gmail, con validación humana antes de toda comunicación externa.

Autor

Tomás Etiennot Sardo

Stack

Make · Airtable · Anthropic Claude · Gmail · Tally

Caso de negocio

KEMET · VIBRATE · PALMA

Repositorio

github.com/tomasetiennot-afk/ecosistema-frees-ia

El proceso que se automatizó

Las tres marcas otorgan entradas de cortesía —*frees*— a personas que aportan difusión o convocatoria. El proceso original era enteramente manual: los pedidos llegaban por mensaje directo de Instagram en formato libre, la evaluación dependía del criterio del momento, y no quedaba registro de a quién se le había dado free, si había asistido, ni cuánto cupo quedaba por evento. El resultado eran decisiones inconsistentes, cupo desbordado y cero trazabilidad.

El sistema construido reemplaza ese circuito por un flujo de dos escenarios encadenados. El primero se dispara solo, normaliza la solicitud, recupera las políticas de la marca desde una base de conocimiento, consulta el historial del solicitante, la evalúa con un modelo de lenguaje contra criterios explícitos y versionados, persiste el resultado con su desglose auditable y frena. El segundo se activa únicamente cuando una persona valida la decisión, y recién ahí se produce la comunicación con el solicitante.

2	15	5	3
ESCENARIOS	NODOS	TABLAS	REGLAS DURAS

Decisión de arquitectura: por qué el trigger no es Instagram

El canal real de entrada era el mensaje directo de Instagram. Se descartó como disparador por dos razones, y la segunda pesa más que la primera. La técnica: la API de mensajería de Instagram no expone un conector utilizable para este caso de uso. La operativa: un pedido en texto libre por DM llega sin estructura, sin constancia y sin consentimiento explícito sobre el uso de los datos, lo que obliga a reconstruir a mano cada solicitud antes de poder procesarla.

La solución adoptada rediseña el proceso antes de automatizarlo: el DM pasa a tener una respuesta automática con un enlace a un formulario público, y el dato nace estructurado en origen. Esto habilita un *webhook* instantáneo sin consumo de operaciones por sondeo, elimina el canal paralelo informal que generaba pedidos sin registro, y permite incluir el aviso de intervención de IA exigido por el marco regulatorio en el mismo punto donde se recolectan los datos.

CRITERIO DE DISEÑO

El formulario estructura los datos duros, pero conserva un campo abierto obligatorio —el *pitch* del solicitante— que es la materia prima real del motor de IA. Sin ese campo, el sistema sería un conjunto de reglas aritméticas y no habría nada que interpretar en lenguaje natural.

Los cinco entregables y dónde están

CRITERIO	PESO	UBICACIÓN
Mapa de arquitectura del sistema	20 %	Sección 1 de este documento
Manual operativo de estructuras de datos	20 %	Sección 2 de este documento
Estrategia de optimización de costos	20 %	Sección 3 de este documento
Malla de seguridad, privacidad y resiliencia	20 %	Sección 4 de este documento
Dashboard de control ejecutivo	20 %	Sección 5 · enlaces públicos

Mapa de arquitectura del sistema

Modelado visual del recorrido completo del dato: disparador de entrada, router de distribución, APIs integradas, nodo de inteligencia artificial, rutas de contingencia y destino final de la información.

Escenario 1 — Intake y scoring

Se dispara con cada solicitud enviada y corre sin intervención humana hasta el punto de validación. Normaliza los datos del formulario, resuelve el evento contra la base, decide por qué rama continuar, inyecta las políticas de la marca en el prompt, evalúa con Claude, persiste el resultado y notifica para aprobación. Dos mecanismos distintos lo protegen: un **router** que bifurca ante datos faltantes, y un **error handler** que captura los fallos de la API de inteligencia artificial.

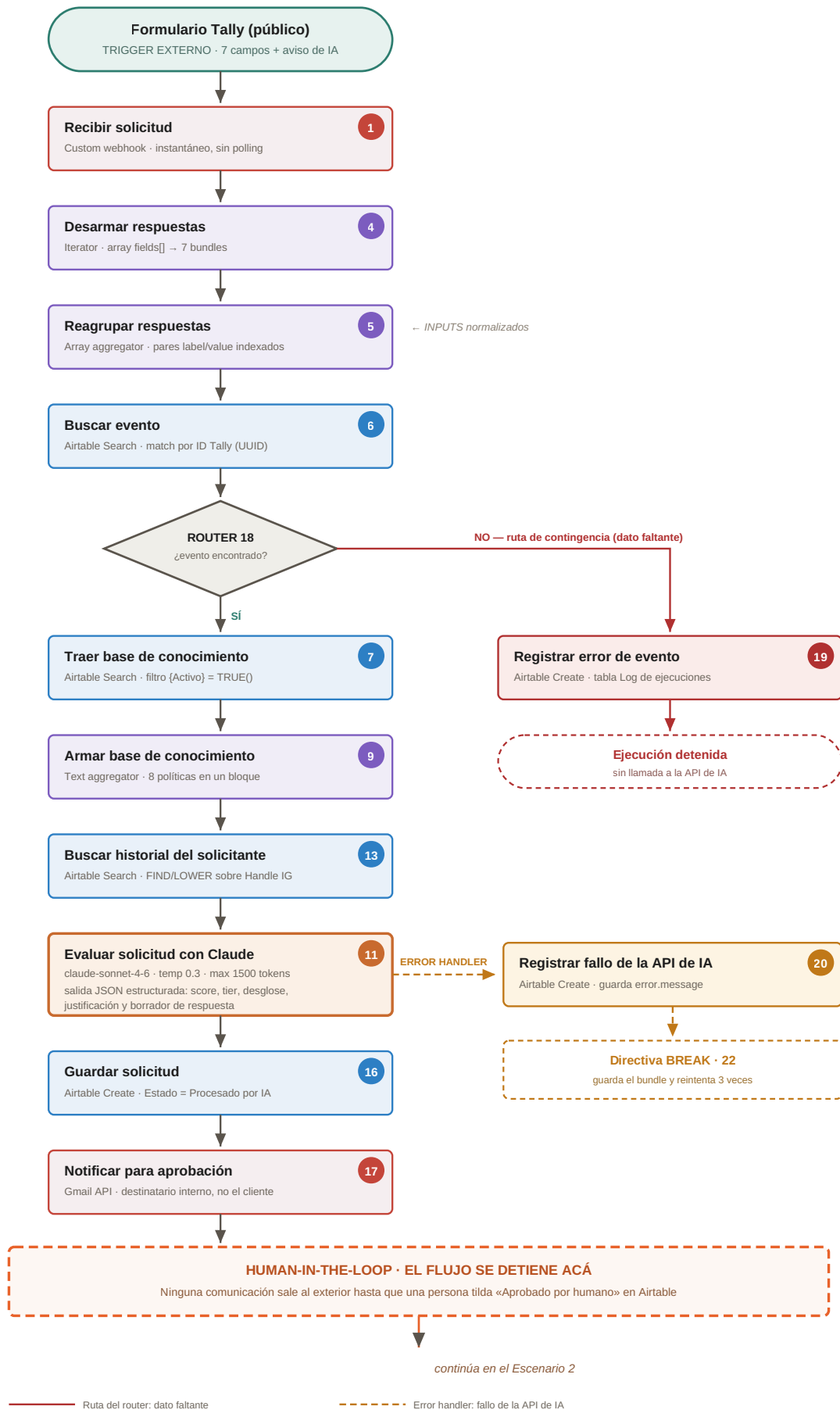


Figura 1. Escenario 1 en Make. Los números corresponden a los IDs reales de módulo del blueprint exportado. Los dos mecanismos de contingencia operan sobre fallos distintos: el router (rojo) desvía cuando el dato de entrada no resuelve, y el error handler (naranja punteado) captura la excepción cuando el módulo de IA devuelve un error de API.

Escenario 2 — Notificación post-aprobación

Es la continuación del circuito. No se dispara por tiempo ni por la creación del registro, sino por la modificación del campo que refleja la decisión humana. El filtro que sigue al trigger cumple una función doble: verifica que efectivamente haya aprobación y descarta los registros ya notificados, lo que evita el envío repetido y el bucle de reentrada.

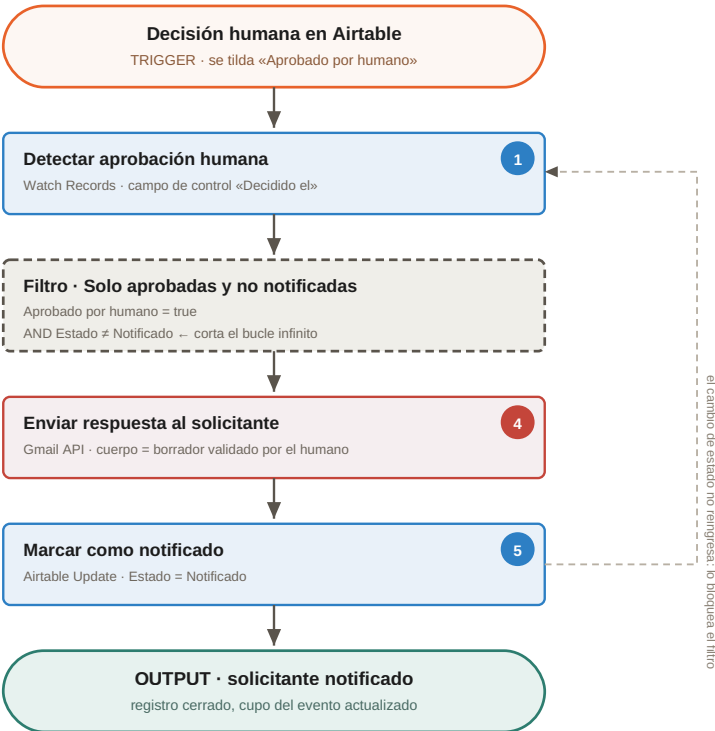


Figura 2. Escenario 2 en Make. La línea punteada de la derecha representa la reentrada potencial que el filtro neutraliza: al actualizar el estado, el trigger vuelve a detectar el registro, pero la condición **Estado ≠ Notificado** lo descarta.

Inventario de nodos

ID	NODO	TIPO	FUNCIÓN EN LA TRAZABILIDAD
1	Recibir solicitud	Custom webhook	Punto de entrada del dato externo
4	Desarmar respuestas	Iterator	Descompone el array anidado del formulario
5	Reagrupar respuestas	Array aggregator	Produce los inputs normalizados e indexados
6	Buscar evento	Airtable Search	Resuelve el evento y trae marca, cupo y otorgados
18	¿Evento encontrado?	Router	Bifurca según exista o no el registro de evento
7	Traer base de conocimiento	Airtable Search	Recupera las políticas activas de evaluación
9	Armar base de conocimiento	Text aggregator	Concatena las políticas en un bloque inyectable
13	Buscar historial	Airtable Search	Recupera solicitudes previas del mismo handle
11	Evaluar con Claude	Anthropic API	Motor de razonamiento · salida JSON estructurada
20	Registrar fallo de la API de IA	Error handler · Airtable Create	Captura la excepción del módulo 11 y la persiste con su mensaje
22	Directiva Break	Error handler · control	Guarda el bundle fallido y programa 3 reintentos
16	Guardar solicitud	Airtable Create	Persiste la evaluación con su desglose
17	Notificar para aprobación	Gmail API	Semáforo de validación manual
19	Registrar error de evento	Airtable Create	Ruta de contingencia · alimenta la tasa de error

Trazabilidad del dato, de punta a punta

Un dato entra como texto en un formulario público y sale como un correo redactado en el tono de la marca. El recorrido completo, con la transformación que sufre en cada salto, es el siguiente: el *payload* del webhook llega como objeto anidado con las siete respuestas dentro de un array; el iterator y el agregador lo aplanan a pares clave-valor direccionables por índice; la búsqueda de evento convierte un identificador opaco en el contexto de negocio —marca, cupo total, frees ya otorgados—; la base de conocimiento aporta las reglas; el historial aporta la reputación del solicitante; el modelo combina las tres fuentes y devuelve una decisión con su desglose; la escritura en Airtable congela esa decisión con estado pendiente; y el segundo escenario la convierte en comunicación sólo después de que una persona la confirme.

TRIGGER INTELIGENTE

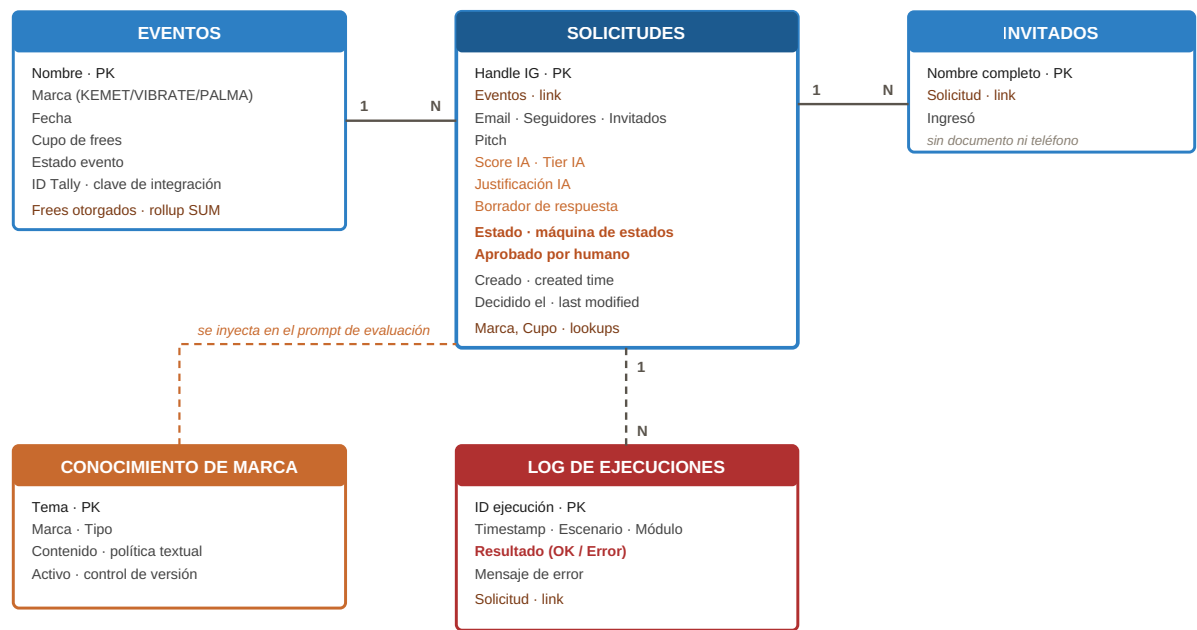
La consigna advierte sobre el consumo innecesario de operaciones. El webhook se eligió precisamente por eso: a diferencia de un *Watch Records* o un disparador programado, no consume operaciones mientras no haya actividad. El escenario sólo corre cuando efectivamente entra una solicitud.

Manual operativo de estructuras de datos

Definición técnica del cerebro relacional en Airtable y de los esquemas JSON de transferencia que viajan entre las integraciones del ecosistema.

Modelo relacional

La base `appIVK7AvA6q0rnQA` contiene cinco tablas. Tres forman la cadena operativa —evento, solicitud, invitados—, una funciona como memoria de criterios y la quinta como registro de salud del sistema. No hay tablas aisladas: cada una se vincula o alimenta a otra.



Línea continua: relación por campo vinculado. Línea punteada: dependencia funcional en tiempo de ejecución.
Naranja: campos escritos por la IA. Rojo: campos que gobiernan la validación humana.

Figura 3. Modelo entidad-relación. `Frees otorgados` es un rollup que suma `Cantidad de invitados` de las solicitudes vinculadas, y es el dato que permite al modelo verificar el cupo disponible antes de evaluar.

Especificación de tablas

Eventos `tblduU8E0nJ6d0qVL`

CAMPO	TIPO	PROPÓSITO
Nombre	Texto de una línea	Clave primaria del evento
Marca	Selección única	KEMET · VIBRATE · PALMA. Determina qué políticas aplican
Fecha	Fecha	Se inyecta en el borrador de respuesta
Cupo de frees	Número	Techo de invitados de cortesía del evento
Estado evento	Selección única	Planificado · Abierto a frees · Cerrado · Realizado
ID Tally	Texto de una línea	UUID de la opción del formulario. Clave de integración estable
Frees otorgados	Rollup SUM	Suma de invitados de solicitudes vinculadas

POR QUÉ EL VÍNCULO ES POR UUID Y NO POR NOMBRE

El formulario devuelve el identificador interno de la opción elegida, no su texto. Se podría traducir con una expresión, pero se optó por almacenar el UUID en la tabla y buscar por él. El motivo es de resiliencia: si mañana el evento se renombra, el identificador no cambia y la integración sigue funcionando. Vincular por coincidencia de texto genera un acoplamiento frágil que se rompe con una tilde o un espacio.

CAMPO	TIPO	PROPÓSITO
Handle IG	Texto de una línea	Identidad del solicitante. Clave de búsqueda del historial
Eventos	Vínculo (1 registro)	Relación con el evento solicitado
Email	Correo electrónico	Canal de respuesta. Único dato de contacto recolectado
Seguidores	Número	Variable de mayor peso del scoring
Cantidad de invitados	Número	Alimenta el rollup de cupo y la regla de tope de grupo
Pitch	Texto largo	Campo abierto. Materia prima del análisis de lenguaje natural
Score IA	Número	Puntaje 0–100 devuelto por el modelo
Tier IA	Selección única	A · B · C · Rechazado
Justificación IA	Texto largo	Explicación variable por variable. Uso interno
Borrador de respuesta	Texto largo	Mensaje propuesto. No se envía sin validación
Estado	Selección única	Pendiente → Procesado por IA → Aprobado/Rechazado → Notificado
Aprobado por humano	Casilla	Semáforo del HITL. Su cambio dispara el escenario 2
Creado	Fecha de creación	Marca temporal de ingreso
Decidido el	Última modificación	Acotado a Estado y Aprobado por humano. Campo de disparo
Marca · Cupo de frees	Búsqueda	Traídos desde Eventos. Evitan un módulo adicional de consulta

Invitados · Conocimiento de marca · Log de ejecuciones

TABLA	CAMPOS	ROL EN EL SISTEMA
Invitados	Nombre completo, Solicitud (link), Ingresó	Registro nominal de acceso. Deliberadamente mínima: alimenta el historial de asistencia sin almacenar documentos
Conocimiento de marca	Tema, Marca, Tipo, Contenido, Activo	Memoria de criterios. Ocho registros atómicos que se recuperan por filtro y se inyectan en el prompt en cada ejecución
Log de ejecuciones	ID ejecución, Timestamp, Escenario, Módulo, Resultado, Mensaje de error, Solicitud (link)	Registro de salud. Recibe escrituras de las dos rutas de contingencia y es la fuente del KPI de tasa de errores

LA BASE DE CONOCIMIENTO COMO RECUPERACIÓN POR FILTRO

Las políticas de evaluación no están escritas dentro del prompt. Viven como registros atómicos en una tabla, cada uno con su campo **Marca** y su **Activo**, y se recuperan en tiempo de ejecución con el filtro `{Activo} = TRUE()` antes de concatenarse e inyectarse en el contexto del modelo. Es una recuperación por filtro sobre base estructurada: cambiar un criterio de negocio se hace editando una celda, sin tocar el escenario ni reescribir el prompt, y desactivar una política es destildar una casilla.

Esquemas JSON de transferencia

Cuatro estructuras gobiernan el movimiento de información entre integraciones. Se documentan con los valores reales capturados durante las pruebas.

1 · Entrada — *payload* del formulario al webhook

El formulario no envía las respuestas como campos planos. Las anida dentro de un array donde cada elemento es un objeto con clave, etiqueta, tipo y valor. Los campos de selección devuelven el valor como array de identificadores, con la tabla de opciones adjunta.

```
{
  "eventId": "b876f2ea-d050-4e1e-babe-5281fbf1a340",
  "eventType": "FORM_RESPONSE",
  "createdAt": "2026-09-10T11:10:00.000Z",
  "data": {
    "responseId": "zEg0l6E",
    "formId": "1Aeqyb",
    "formName": "Solicitud de free",
    "submissionPdfUrl": "https://... accessToken=...", // descartado por diseño
    "fields": [
      { "key": "question_I7EWRW", "label": "Nombre completo",
        "type": "INPUT_TEXT", "value": "Martina Roldán" },
      { "key": "question_RJ9YLJ", "label": "Usuario de instagram",
        "type": "INPUT_TEXT", "value": "@martu.rolدان" },
      { "key": "question_v09Lyv", "label": "Email",
        "type": "INPUT_EMAIL", "value": "[...]@gmail.com" },
      { "key": "question_KJjDIA", "label": "Cantidad de seguidores",
        "type": "INPUT_NUMBER", "value": 2500 },
      { "key": "question_xPW0Xr", "label": "Evento",
        "type": "DROPDOWN",
        "value": ["02e426fc-6f90-4286-bc80-03e0f41d493b"],
        "options": [
          { "id": "02e426fc-...", "text": "ACAI" },
          { "id": "64218c74-...", "text": "JOGO" },
          { "id": "85cbc762-...", "text": "Mar y Fuego" }
        ]
      },
      { "key": "question_p72WBq", "label": "Cantidad de personas totales",
        "type": "INPUT_NUMBER", "value": 2 },
      { "key": "question_1Jav7Q", "label": "Motivo de pedido de free",
        "type": "TEXTAREA", "value": "Subo tres stories etiquetando..." }
    ]
  }
}
```

2 · Intermedio — inputs normalizados

El iterator recorre `fields[]` y el agregador devuelve un array plano de pares etiqueta-valor en orden estable. A partir de aquí todo el escenario direcciona por índice, con referencias legibles del tipo `5.array[4].value`.

```
{
  "array": [
    { "label": "Nombre completo", "value": "Martina Roldán" },
    { "label": "Usuario de instagram", "value": "@martu.rolan" },
    { "label": "Email", "value": "[...].gmail.com" },
    { "label": "Cantidad de seguidores", "value": "2500" },
    { "label": "Evento", "value": ["02e426fc-..."] },
    { "label": "Cantidad de personas totales", "value": "2" },
    { "label": "Motivo de pedido de free", "value": "Subo tres stories..." }
  ]
}
```

3 · Contrato de salida del modelo

El prompt fuerza una respuesta JSON estricta, sin *markdown* ni texto envolvente. El conector de Anthropic la entrega ya parseada bajo `jsonResponse`, lo que elimina la necesidad de un módulo de parseo adicional. El `desglose` no es decorativo: es el mecanismo de trazabilidad que permite auditar qué variable infló o hundió un puntaje.

```
{
  "score": 68,
  "tier": "B",
  "desglose": {
    "seguidores": 44, // sobre 55
    "contraprestacion": 18, // sobre 25
    "historial": 8, // sobre 15
    "grupo": 5 // sobre 5
  },
  "justificacion": "Con 1.600 seguidores cae en el rango 1.400-1.999, se le asignan 44 puntos. La contraprestación es sólida: dos stories etiquetadas con flyer (verificable), grupo de difusión de 200 personas (menciona el grupo pero no adjunta link ni captura, así que se puntúa con moderación) y 4 personas que compran entrada (concreto y valioso), lo que justifica 18 sobre 25. Sin historial previo arranca en punto medio-bajo, 8 sobre 15.",
  "borrador_respuesta": "Dale Sofía, quedás en la lista para el 18 de enero. Sos vos + 2, pasame los nombres completos de cada uno antes del 16 así los dejo cargados. Nos vemos ahí."
}
```

4 · Escritura en la base

El módulo de creación de registro combina tres orígenes: los inputs normalizados del formulario, el identificador de registro devuelto por la búsqueda de evento —que materializa el vínculo relacional— y los campos producidos por el modelo. El estado inicial es siempre `Procesado por IA`, nunca aprobado.

CAMPO DESTINO	ORIGEN	TIPO DE DATO
Handle IG	5.array[2].value	string
Email	5.array[3].value	string · email
Seguidores	5.array[4].value	number
Cantidad de invitados	5.array[6].value	number
Pitch	5.array[7].value	string · long text
Eventos	6.id	array de record ID
Score IA	11.jsonResponse.score	number
Tier IA	11.jsonResponse.tier	enum
Justificación IA	11.jsonResponse.justificacion	string
Borrador de respuesta	11.jsonResponse.borrador_respuesta	string
Estado	constante Procesado por IA	enum
Aprobado por humano	vacío · lo completa una persona	boolean

Especificación del esquema en lenguaje natural

El modelo relacional se diseñó a partir de la siguiente especificación, redactada antes de construir la base y utilizable como instrucción de generación asistida en Airtable. Se documenta aquí porque define el contrato de datos con independencia de la herramienta que lo materialice:

Creá una base para gestionar solicitudes de entradas de cortesía de una productora que opera tres marcas de eventos.

Necesito una tabla Eventos con nombre, marca (selección única: KEMET, VIBRATE, PALMA), fecha, cupo de frees numérico, estado del evento y un campo de texto para el identificador del formulario externo.

Una tabla Solicitudes vinculada a Eventos con: handle de Instagram, email, seguidores, cantidad de invitados, un campo de texto largo para el pitch, campos numéricos y de texto para el resultado de la evaluación automática, un campo de estado con las opciones Pendiente, Procesado por IA, Aprobado, Rechazado y Notificado, una casilla de aprobación manual, un campo de fecha de creación y uno de última modificación limitado a los campos de estado y aprobación.

Una tabla Invitados vinculada a Solicitudes, con nombre completo y una casilla de ingreso. No incluyas documento, teléfono ni fecha de nacimiento.

Una tabla Conocimiento de marca con tema, marca, tipo de criterio, contenido de texto largo y una casilla de activo.

Una tabla Log de ejecuciones con identificador, escenario, módulo, resultado (OK o Error), mensaje y vínculo a la solicitud.

En Eventos agregá un rollup que sume la cantidad de invitados de las solicitudes vinculadas cuyo estado sea Notificado.

SOBRE LA INSTRUCCIÓN DE EXCLUSIÓN

La frase «no incluyas documento, teléfono ni fecha de nacimiento» forma parte de la especificación a propósito. Un generador asistido, librado a su criterio, propone esos campos por defecto para una tabla de invitados a un evento. La minimización de datos se declara en el diseño del esquema, no se corrige después.

Estrategia de optimización de costos y recursos

Matriz de decisión sobre qué modelo se asigna a cada tarea, con el consumo real medido en producción y la certificación del ahorro frente a una implementación sin criterio de costos.

Consumo real medido

El punto de partida no es una estimación: es el consumo registrado por el conector de Anthropic en una ejecución completa del escenario 1, con una solicitud de longitud representativa.

MÉTRICA	TOKENS	COMPOSICIÓN
Tokens de entrada	2.158	System prompt + base de conocimiento + contexto de evento + solicitud
— porción invariante	≈ 1.800	System prompt, 8 políticas, plantilla de salida JSON
— porción variable	≈ 358	Datos del evento, del solicitante y el pitch
Tokens de salida	304	JSON con score, tier, desglose, justificación y borrador
Total por evaluación	2.462	Una solicitud procesada de punta a punta

SUPUESTOS DE VOLUMEN

Los cálculos de esta sección se proyectan sobre **40 solicitudes por evento y 4 eventos mensuales entre las tres marcas, es decir 160 evaluaciones por mes**. Es una estimación de referencia declarada, no una medición histórica. El sistema registra el volumen real en la tabla de solicitudes, de modo que los valores se recalculan sustituyendo esta cifra por el dato observado.

Matriz de decisión por tarea

El ecosistema no resuelve una única tarea homogénea. Tiene tres cargas de trabajo con perfiles distintos de complejidad, urgencia y volumen, y asignarles el mismo modelo sería tan caro como innecesario.

TAREA	PERFIL	ASIGNACIÓN	JUSTIFICACIÓN
A · Evaluación de solicitud <i>implementada</i>	Lectura densa · interpretación de lenguaje natural · latencia baja requerida	Claude Sonnet 4.6 <code>temp 0.3</code> <code>max_tokens 1500</code>	Debe leer un pitch en texto libre, contrastarlo contra ocho políticas, detectar si una contraprestación es verificable o retórica, y redactar una respuesta en el registro de voz de la marca. Es la única tarea del sistema que justifica un modelo de gama alta: un error acá se convierte en un mensaje mal escrito enviado a un cliente.
B · Validación de completitud <i>implementada</i>	Mecánica · determinística · sin ambigüedad	Sin IA campos obligatorios + router	Verificar que exista el evento y que los campos estén completos no requiere interpretación. Se resuelve con validación del formulario y una condición del router, a costo cero. Delegar esto a un modelo económico seguiría siendo más caro que no delegarlo.
C · Reconciliación post-evento <i>proyectada</i>	Masiva · diferible · tolera latencia de horas	Claude Haiku 4.5 vía API de lotes	Al cerrar cada evento hay que cruzar la lista de invitados contra el ingreso real, recalcular la reputación de cada solicitante y producir un resumen por marca. Son decenas de registros homogéneos, sin urgencia: el caso de uso exacto de un procesamiento por lotes con el modelo más económico.

Comparativa de modelos para la tarea A

Precios de referencia por millón de tokens. La columna de costo mensual proyecta las 160 evaluaciones con el consumo medido.

MODELO	ENTRADA	SALIDA	COSTO MENSUAL	ADECUACIÓN
Claude Opus	\$15,00	\$75,00	\$8,83	Sobredimensionado. No mejora una tarea acotada por reglas explícitas
Claude Sonnet 4.6	\$3,00	\$15,00	\$1,77	Elegido. Sostiene la lectura del pitch y la redacción con voz de marca
Claude Haiku 4.5	\$1,00	\$5,00	\$0,59	Viable en scoring, más flojo redactando la respuesta al cliente
GPT-4o-mini	\$0,15	\$0,60	\$0,08	El más barato, pero el borrador pierde el registro rioplatense

EL CRITERIO QUE ORDENA LA DECISIÓN

La diferencia entre el modelo más caro y el más barato es de ocho dólares y setenta y cinco centavos al mes. A este volumen, el costo no es la variable que decide: lo que decide es que el borrador salga escrito como lo escribiría una persona de la producción. Un mensaje que suena a formulario quema el vínculo con alguien que difunde los eventos, y ese costo no aparece en ninguna factura de API. Se eligió Sonnet como punto de equilibrio: cinco veces más barato que Opus sin perder calidad de redacción. El salto a Haiku o a un modelo mini se reevaluará cuando el volumen mensual supere las mil evaluaciones, momento en que la diferencia empieza a ser material.

Certificación del ahorro

El ahorro se mide contra una línea base explícita: la implementación que resultaría de no tomar ninguna decisión de costos, es decir, asignar el modelo más capaz a todas las tareas y ejecutarlas todas en tiempo real.

CARGA DE TRABAJO	LÍNEA BASE (OPUS, TIEMPO REAL)	STACK IMPLEMENTADO	AHORRO
A · Evaluación de solicitud 160 evaluaciones/mes	\$8,83	\$1,77	79,9 %
B · Validación de completitud 160 verificaciones/mes	\$1,32	\$0,00	100 %
C · Reconciliación post-evento 160 registros/mes	\$5,28	\$0,18	96,6 %
Total mensual	\$15,43	\$1,95	87,4 %

El ahorro consolidado es del **87,4 %**, holgadamente por encima del umbral del 50 %. Tres decisiones lo producen, en orden de contribución: no ejecutar con IA lo que se resuelve con una condición lógica, delegar el trabajo masivo y diferible a la API de lotes con el modelo económico, y reservar el modelo de gama alta únicamente para la tarea donde la calidad del texto tiene consecuencias comerciales.

Una optimización que se evaluó y se descartó

El almacenamiento en caché del prompt parece la optimización obvia: de los 2.158 tokens de entrada, unos 1.800 son idénticos en cada llamada —el system prompt, las ocho políticas, la plantilla de salida— y la lectura desde caché cuesta una fracción del precio de entrada estándar.

No se implementó, y el motivo es de patrón de tráfico. El caché tiene una vigencia de pocos minutos, y las solicitudes de free no llegan agrupadas: llegan dispersas a lo largo de días. Con ese perfil, casi ninguna llamada encontraría el caché vigente, y cada escritura de caché tiene un recargo sobre el precio de entrada. El resultado neto sería un costo mayor, no menor. La técnica queda anotada para el escenario en que el volumen justifique procesar solicitudes en tandas —por ejemplo, un lote nocturno al cerrar la convocatoria de un evento—, que es cuando el caché efectivamente rinde.

CRITERIO GENERAL

Aplicar una técnica de optimización porque está disponible, sin verificar que el patrón de uso la habilite, es una forma de encarecer el sistema creyendo que se lo abarata. La decisión de no usar caché es tan parte de la estrategia de costos como la decisión de usar lotes.

Control de operaciones del orquestador

El costo de la plataforma de automatización se mide en operaciones consumidas, y ahí el diseño del disparador pesa más que el del prompt.

CONCEPTO	OPERACIONES	OBSERVACIÓN
Escenario 1 · por solicitud	≈ 18	Incluye los 8 registros devueltos por la consulta a la base de conocimiento
Escenario 2 · por aprobación	3	Trigger, envío y actualización de estado
Ruta del router · por fallo	≈ 7	Se corta antes de la IA: no consume la llamada al modelo
Error handler · por fallo de API	≈ 15	Llega hasta el módulo de IA, registra el error y reintenta
Proyección mensual · 160 solicitudes	≈ 3.400	Requiere plan intermedio; el plan gratuito de 1.000 operaciones es insuficiente

Decisiones que contienen el consumo de operaciones

- **Disparador por webhook en lugar de sondeo.** Un *Watch Records* o un disparador programado consumen operaciones aunque no haya novedades. El webhook sólo se activa cuando entra una solicitud real: con cuatro eventos al mes, la diferencia frente a un sondeo cada quince minutos es de miles de operaciones.
- **Corte temprano en la ruta del router.** Si el evento no se resuelve, la ejecución se detiene antes de las consultas y —sobre todo— antes de pagar la llamada al modelo de lenguaje.
- **Reintento acotado en el error handler.** La directiva de reintento está limitada a tres intentos espaciados, no a un bucle abierto: un fallo persistente de la API no puede consumir la cuota mensual reintentando indefinidamente.
- **Filtro anti-reentrada en el escenario 2.** Sin la condición `Estado ≠ Notificado`, la actualización del registro volvería a disparar el trigger y cada solicitud generaría envíos en cadena. El filtro no es sólo una protección funcional: es contención de costo.
- **Campos de búsqueda en lugar de consultas adicionales.** La marca y el cupo del evento se traen como campos vinculados desde la tabla de eventos, evitando un módulo de consulta por ejecución.

Elección de plataforma: no-code sobre low-code

La orquestación se resolvió en Make, una plataforma no-code, y no en n8n. La justificación tiene tres ejes. En **velocidad**, todas las integraciones requeridas —formulario, base de datos, modelo de lenguaje, correo— tienen conector nativo, de modo que no hay nada a medida que construir. En **flexibilidad**, la única lógica no trivial del sistema es la bifurcación del router, la normalización del payload anidado y el manejo de excepciones, y las tres se resuelven con módulos estándar: no aparece ningún requerimiento que obligue a escribir código. En **límites técnicos**, el volumen proyectado se ubica holgadamente dentro de un plan intermedio, sin acercarse a los topes de ejecución.

El umbral para migrar a low-code está identificado: si el sistema necesitara integrarse contra la plataforma de ticketing —que no expone un conector estándar— o computar reputación con una lógica de ponderación temporal, el terreno pasaría a ser el de n8n. Hoy, adoptar low-code agregaría curva de aprendizaje y mantenimiento de código propio sin resolver ningún problema existente.

Malla de seguridad, privacidad y resiliencia

Políticas de minimización aplicadas al flujo, configuración de las rutas de contingencia y de los manejadores de error, y justificación del punto donde se insertó el semáforo de validación manual.

Minimización de datos

El principio rector es que un dato que no se recolecta no se puede filtrar, no hay que custodiarlo y no hay que borrarlo. La decisión se tomó en el diseño del esquema, no como corrección posterior: el formulario pide exactamente siete campos y cada uno tiene una función identificable en el proceso.

DATO	ESTADO	FUNDAMENTO
Nombre completo	Se recolecta	Necesario para personalizar la respuesta y armar la lista de acceso
Usuario de Instagram	Se recolecta	Clave de identidad del solicitante y única vía de consulta del historial
Email	Se recolecta	Canal de respuesta. Es el único dato de contacto del sistema
Cantidad de seguidores	Se recolecta	Declarado por el solicitante. Variable principal del scoring
Evento, invitados, pitch	Se recolecta	Objeto de la solicitud y materia prima de la evaluación
Documento de identidad	No se recolecta	El control de acceso del establecimiento lo requiere, pero se gestiona por fuera del sistema y no persiste en la base. La tabla de invitados guarda nombre y estado de ingreso, nada más
Teléfono	No se recolecta	El contacto es por correo. Un segundo canal sería un dato personal adicional sin función
Fecha de nacimiento	No se recolecta	La validación de edad ocurre en la puerta, con documento, fuera del circuito digital
Foto de perfil, seguidos en común	No se recolecta	Extraer datos del perfil del solicitante para evaluarlo excedería aquello para lo que entregó su información

Descarte deliberado dentro del propio payload

El formulario envía más información de la que el sistema necesita. Entre los campos del *payload* viajan dos direcciones — `submissionPdfUrl` y `submissionPreviewUrl` — que incluyen un token de acceso incrustado en la propia URL y permiten recuperar la respuesta completa sin autenticación. Ninguna de las dos se mapea a la base ni se registra en ningún log. La minimización no se aplica sólo a lo que se pide: también a lo que llega sin haber sido pedido.

Equidad algorítmica

El sistema evalúa personas, y por lo tanto puede discriminarlas. El riesgo no es hipotético: un criterio de evaluación intuitivo, trasladado sin filtro a un algoritmo, convierte un sesgo ocasional en una regla aplicada de forma sistemática a cada solicitud, con la apariencia de objetividad que da un puntaje numérico.

La contención opera en dos niveles. El primero es el diseño del modelo de scoring: las cuatro variables son alcance declarado, contraprestación ofrecida, historial verificado y tamaño del grupo. Todas miden comportamiento o resultado, ninguna mide atributos de la persona. El segundo es una instrucción explícita en el *system prompt*, que actúa como barrera incluso si el pitch del solicitante introdujera información sensible por su cuenta:

```
// Fragmento del system prompt · módulo 11
```

```
NUNCA evalúes ni menciones género, apariencia física, edad,
nacionalidad ni ningún atributo personal del solicitante. Solo
evaluás las variables definidas en la base de conocimiento.
```

TRANSPARENCIA VERIFICABLE

El campo `desglose` del JSON de salida obliga al modelo a declarar cuántos puntos asignó a cada variable. Eso convierte la pregunta «¿por qué este pedido se rechazó?» en una consulta a la base, no en una conjetura. Sin desglose, el sistema sería una caja negra que emite un número; con desglose, cada decisión es auditable variable por variable y cualquier sesgo emergente resulta detectable al comparar puntajes.

Encuadre regulatorio

Clasificación bajo la IA Act

El sistema se clasifica como de **riesgo limitado**. No decide sobre acceso al empleo, crédito, educación ni servicios esenciales: decide sobre la asignación de entradas de cortesía a un evento privado. No cae en ninguna de las categorías de alto riesgo ni en las prácticas prohibidas.

La obligación central de ese nivel es la transparencia: la persona tiene que saber que hay una inteligencia artificial interviniendo. Se cumple con un aviso en el propio formulario, en el punto donde se recolectan los datos y antes de que se envíen:

```
// Texto visible al pie del formulario público
```

```
Tu solicitud se evalúa con asistencia de inteligencia artificial y la
decisión final la toma una persona del equipo. Usamos tus datos solo
para gestionar este pedido.
```

La segunda oración cubre la limitación de finalidad exigida por el GDPR: el dato se entrega para gestionar una solicitud concreta y no se reutiliza para otra cosa. En particular, los correos recolectados no alimentan ninguna lista de difusión: hacerlo requeriría un consentimiento específico e independiente que el formulario no solicita.

Retención y derecho de supresión

La base conserva las solicitudes porque el historial de asistencia es una de las cuatro variables del scoring: borrar el registro de quien confirmó y no asistió eliminaría precisamente la información que el sistema necesita. La política definida es de conservación acotada al valor operativo —el ciclo de eventos de la temporada— con depuración posterior de los datos de contacto, manteniendo el historial en forma agregada. Un pedido de supresión se resuelve eliminando el registro de la solicitud y sus invitados vinculados, sin que ello afecte el funcionamiento del flujo.

Resiliencia · rutas de contingencia y manejadores de error

Un flujo que se rompe ante el primer dato inconsistente no es un flujo automatizado: es una tarea manual con pasos adicionales. El sistema distingue dos clases de fallo y las trata con mecanismos distintos. Los **datos que no resuelven** se desvían con un router antes de consumir recursos. Las **excepciones de una API externa** se capturan con un error handler, porque un router no puede interceptar un módulo que devuelve error: la ejecución se detendría antes de alcanzarlo.

Ruta 1 · Evento no resuelto — router

Si la búsqueda del evento no devuelve registro —identificador ausente, evento eliminado, formulario desactualizado—, el flujo no continúa hacia la evaluación. Toma la segunda rama del router, escribe un registro en la tabla de log con el escenario, el módulo donde falló, el identificador de ejecución y el mensaje con el valor que no pudo resolverse, y termina.

CONDICIÓN DEL ROUTER	RAMA	COMPORTAMIENTO
<code>{{6.id}}</code> existe	Evento encontrado	Continúa hacia base de conocimiento, historial, evaluación y persistencia
<code>{{6.id}}</code> no existe	Evento no encontrado	Escribe en Log de ejecuciones y detiene la ejecución

El orden de los módulos es deliberado: el router está inmediatamente después de la búsqueda de evento y antes de toda la cadena de consultas y de la llamada al modelo. Una solicitud que no se puede resolver no consume tokens ni operaciones de integración. La contención de fallos y la contención de costos son, en este punto, la misma decisión de arquitectura.

EVIDENCIA · PRUEBA DEL CAMINO INFELIZ

Se eliminó el identificador de integración de un evento y se envió una solicitud para ese evento. La ejecución tomó la rama de contingencia: la rama principal quedó en cero, el módulo de registro de error se ejecutó correctamente y se creó la fila correspondiente en la tabla de log, con resultado **Error**, módulo **Buscar evento** y el identificador no resuelto en el mensaje. Ninguna llamada a la API de inteligencia artificial se produjo. Capturas: `caso4-error-make.png`, `caso4-error-log.png`.

Ruta 2 · Fallo de la API de inteligencia artificial — error handler

Es el manejador de excepciones propiamente dicho, y cubre el modo de falla que no depende del dato de entrada sino del servicio externo: límite de tasa excedido, sobrecarga del proveedor, agotamiento de crédito, tiempo de espera agotado o parámetro rechazado. Sin este manejador, cualquiera de esos casos detendría el escenario dejando la solicitud sin evaluar y sin registro de lo ocurrido.

El manejador está asociado al módulo de evaluación y se compone de dos nodos. El primero escribe la excepción en la tabla de log capturando el mensaje que devuelve la API en la variable `error.message`, de modo que el registro no dice «falló» sino por qué falló. El segundo es la **directiva Break**, que preserva el *bundle* fallido y programa su reintento.

NODO	CONFIGURACIÓN	EFEECTO
Registrar fallo de la API de IA	Airtable Create · Log de ejecuciones	Persiste identificador de ejecución, escenario, módulo, resultado Error y el mensaje devuelto por la API
Directiva Break	3 reintentos · intervalo de 15 minutos	Guarda la ejecución como incompleta y la reintenta automáticamente. La solicitud no se pierde

POR QUÉ BREAK Y NO RESUME

La directiva alternativa, *Resume*, continúa el flujo sustituyendo la salida fallida por un valor de reemplazo. Aplicada acá, obligaría a inventar un JSON con un score y un tier ficticios que se escribirían en la base como si fueran una evaluación real: un dato falso persistido, indistinguible de uno legítimo. Break evita eso por completo. Los fallos de una API de inferencia son mayoritariamente transitorios —picos de carga, límites de tasa—, de modo que preservar el bundle y reintentarlo unos minutos después resuelve el caso sin degradar la calidad del dato. El reintento está acotado a tres intentos, no es un bucle abierto.

Ruta 3 · Incompatibilidad de tipo en la escritura

Detectada durante la construcción: un campo de la base configurado con un tipo que no correspondía al dato entrante provocaba el rechazo de la escritura y la interrupción del flujo con un error de validación de parámetros. Se corrigió en el esquema, y el episodio dejó una lección incorporada al diseño: los tipos de campo son un contrato entre el orquestador y la base, y una discrepancia silenciosa ahí detiene el sistema aunque toda la lógica previa sea correcta.

Ruta 4 · Reentrada y bucle infinito

El escenario 2 se dispara ante la modificación de un registro, y su último módulo modifica ese mismo registro. Sin protección, cada aprobación generaría una cadena de ejecuciones que consumiría la cuota de operaciones y enviaría correos repetidos al solicitante. El filtro posterior al trigger lo impide con dos condiciones encadenadas:

```
Aprobado por humano = true
AND
Estado ≠ Notificado
```

La primera condición garantiza que sólo avancen las solicitudes efectivamente validadas. La segunda es la que corta el ciclo: cuando el último módulo escribe el estado **Notificado**, el trigger vuelve a detectar el registro, pero la condición lo descarta. Adicionalmente, el campo de última modificación que gobierna el disparador está acotado a los campos de estado y aprobación, de modo que editar cualquier otro dato del registro no reactiva el flujo.

EVIDENCIA · EL FILTRO PROBADO EN LAS DOS DIRECCIONES

Un filtro que bloquea todo también produciría cero envíos, de modo que verificar sólo el bloqueo no demuestra nada. Se probaron los dos sentidos. Con una solicitud ya notificada, el trigger la detectó y el filtro la descartó: contador en cero, sin correo enviado. Con una solicitud aprobada y pendiente, el filtro la dejó pasar, el correo salió y el registro quedó en estado **Notificado**. Capturas: *antibucle-filtro-bloquea.png*, *antibucle-filtro-pasa.png*.

Trazabilidad

Tres mecanismos permiten reconstruir qué ocurrió en cada ejecución. El historial nativo del orquestador conserva entrada y salida de cada módulo. La tabla de log recibe tanto los fallos de negocio del router como las excepciones de API del error handler, cada uno identificado por su módulo de origen. Y el campo de desglose almacena la composición del puntaje de cada evaluación, lo que permite auditar no sólo si el sistema funcionó, sino si decidió bien.

El punto de validación humana

El semáforo manual se ubica **después de la evaluación y antes de toda comunicación con el exterior**. El escenario 1 termina redactando un borrador y enviando una notificación interna; no le escribe a nadie más. El escenario 2, el único que contacta al solicitante, no tiene disparador propio: sólo puede arrancar si una persona tildó la casilla de aprobación.

Por qué exactamente ahí

- **Es el último punto reversible.** Un puntaje equivocado se corrige editando un registro. Un mensaje enviado a un cliente no se puede retirar.
- **Es donde una alucinación tendría consecuencia externa.** El modelo redacta texto libre dirigido a una persona real, usando su nombre y datos del evento. Cualquier invención —una fecha errónea, una condición que nadie ofreció— saldría de la organización sin filtro.
- **La decisión tiene efecto económico.** Aprobar un free consume cupo y desplaza una entrada paga. El sistema propone; la asignación la decide una persona.
- **Preserva la relación comercial.** El destinatario no es un ticket anónimo: es alguien con quien la productora sostiene un vínculo. Una negativa mal calibrada tiene costo más allá del evento en cuestión.

La notificación de aprobación le llega al responsable con el puntaje, el tier, la justificación completa del modelo y el borrador propuesto. La validación no es un clic ciego: es una revisión con toda la información a la vista.

SOBRE LA APARENTE CONTRADICCIÓN CON LA AUTONOMÍA EXIGIDA

El requisito de que el flujo opere sin intervención manual y el requisito de validación humana no se contradicen: se resuelven separando el ciclo en dos tramos. El primero es íntegramente automático —desde que llega la solicitud hasta que queda evaluada, persistida y notificada— y no requiere que nadie intervenga para funcionar. El segundo tramo tiene por disparador la decisión humana, que es precisamente el control que se quiere preservar.

Gestión de credenciales

Cuatro credenciales sostienen el ecosistema: el token de acceso personal de la base de datos, la clave de API del proveedor de inteligencia artificial, la autorización del servicio de correo y la dirección secreta del webhook. Todas residen en el almacén de conexiones del orquestador y ninguna aparece en el cuerpo de los módulos, en los prompts ni en los registros de ejecución. El token de la base se emitió con permisos acotados a las operaciones que el flujo necesita y con alcance restringido a esta única base. Los identificadores de base y tabla que figuran en esta documentación no son credenciales: sin token asociado no otorgan acceso. Los archivos de blueprint publicados en el repositorio conservan únicamente los identificadores internos de conexión de la plataforma, no las claves.

Limitaciones declaradas

Dos aspectos del sistema están implementados de forma parcial y se documentan antes de que un revisor los descubra:

Historial de asistencia. El módulo que recupera las solicitudes previas del solicitante está construido y funciona, y la política de evaluación define una regla dura sobre inasistencias reiteradas. Sin embargo, los contadores de asistencias y ausencias que se inyectan en el prompt están hoy fijados en cero. El motivo no es una omisión de diseño sino la ausencia de datos: la asistencia se registra después del evento, marcando el ingreso real en la tabla de invitados, y a la fecha de esta entrega el sistema todavía no atravesó un ciclo completo de evento realizado. La relación existe, el campo existe y la regla está escrita; lo que falta es la primera temporada de operación que la alimente.

Exposición del panel público. Las vistas compartidas del dashboard son de acceso público y exponen los datos de las solicitudes en modo lectura, incluidos los correos. Es aceptable en el estado actual, donde la base contiene exclusivamente registros de prueba. Antes de operar con solicitudes reales corresponde ocultar el campo de correo en la vista compartida, dejando visibles el handle, el puntaje, el tier y el estado, que es todo lo que el monitoreo necesita.

Dashboard de control ejecutivo

Panel centralizado que permite monitorear la salud del ecosistema y los indicadores de operación sin entrar a la base de datos ni al orquestador.

Acceso

VISTA	ACCESO	ENLACE
Panel de control Interface con KPIs y gráfico	Interno	Captura incluida en el repositorio. La publicación web de interfaces requiere plan de pago; el monitoreo público se sirve por las vistas compartidas
Solicitudes agrupada por estado	Público	airtable.com/appIVK7AvA6qOrnQA/shrxQ5B6QKmlZdJVY
Log de ejecuciones agrupada por resultado	Público	airtable.com/appIVK7AvA6qOrnQA/shr3Exf9iuTwaZ3S9

Indicadores

El panel responde cuatro preguntas operativas y una de salud técnica, que son las que un responsable necesita para saber si el sistema está funcionando y si hay algo esperando por él.

INDICADOR	CÁLCULO	QUÉ PERMITE DECIDIR
Solicitudes totales	Recuento de la tabla Solicitudes	Volumen de demanda de cortesías
Frees resueltos	Recuento con Estado = Notificado	Cuántas solicitudes completaron el ciclo
Pendientes de aprobación	Recuento con Estado = Procesado por IA	Cola de trabajo humano. Si crece, el cuello de botella es la validación
Errores registrados	Recuento del Log con Resultado = Error	Salud técnica. Incluye tanto los desvíos del router como las excepciones de API
Tasa de error	Errores ÷ (solicitudes + errores)	Fiabilidad del ecosistema en el período
Distribución por tier	Gráfico de barras sobre Tier IA	Calibración del scoring. Si todo cae en un tier, el modelo no discrimina

POR QUÉ LA DISTRIBUCIÓN POR TIER NO ES DECORATIVA

Es el indicador que detecta si el sistema dejó de evaluar y empezó a repetir. Un histograma concentrado en un único tier señala que los criterios están mal calibrados o que el modelo perdió capacidad de discriminar entre solicitudes distintas. Es el control de calidad de la decisión automatizada, no una estadística de presentación.

ESTADO AL MOMENTO DE LA ENTREGA

Solicitudes registradas con sus evaluaciones completas, la mayoría notificadas, una en espera de validación y dos errores capturados por las rutas de contingencia. La distribución de tiers muestra casos en B y en C, lo que confirma que el scoring produce resultados diferenciados con entradas distintas. Captura: [dashboard-panel-control.png](#).

Prueba de estrés

Ejecuciones con entradas diseñadas para recorrer los distintos caminos de la lógica, incluido el camino infeliz y el fallo del servicio externo.

#	ENTRADA	QUÉ VERIFICA	RESULTADO
1	2.500 seguidores · 2 invitados · pitch con tres contraprestaciones verificables	Camino de perfil alto. Techo de la escala de seguidores	Evaluación completa · tier alto · borrador de confirmación
2	1.500 seguidores · 3 invitados · pitch de contraprestación parcial	Rango intermedio. Penalización por contraprestación no verificable	Tier intermedio · borrador de confirmación
3	700 seguidores · 2 invitados · pitch sin compromiso concreto	Aplicación de la regla de contraprestación no verificable al piso del rango	Tier bajo · borrador de negativa sin desglosar puntaje
4	Solicitud para un evento con su identificador de integración eliminado	Camino infeliz. Ruta del router ante dato que no resuelve	Rama principal en cero · registro creado en Log · sin llamada a la IA
5	2.200 seguidores · 8 invitados · pitch con grupos de difusión no verificables	Tensión entre perfil alto y grupo abusivo. Regla de tope de tamaño de grupo	Score 38 · tier C · borrador que solicita reducir el grupo y adjuntar prueba de difusión
6	Parámetro inválido forzado en el módulo de evaluación	Fallo de API. Error handler y directiva Break	Excepción capturada · registro en Log con el mensaje de la API · bundle preservado para reintento

El historial completo de corridas queda disponible en el registro de ejecuciones del orquestador, con fecha, hora, estado y operaciones consumidas por cada una. *Captura:* [historial-ejecuciones-make.png.png](#).

Hallazgo del caso 5 y corrección aplicada

La quinta prueba expuso un defecto real del modelo de scoring. En su primera ejecución, una solicitud de ocho invitados obtuvo 56 puntos y tier B —es decir, aprobación sugerida— pese a que la política de la casa considera que a partir de seis invitados el free deja de ser selectivo. La causa era aritmética: el tamaño del grupo pesa apenas 5 puntos sobre 100, de modo que aun puntuando el mínimo no alcanza a contrapesar un perfil de alcance elevado. Una variable de bajo peso no puede vetar por sí sola.

La corrección no fue subir su peso, porque eso habría alterado el equilibrio general del scoring para resolver un caso borde. Se implementó como **tercera regla dura**, evaluada antes del puntaje: a partir de seis invitados el tier queda topeado en C con independencia del score obtenido, y el borrador propone reducir el grupo o compensar con una contraprestación verificable mayor. El puntaje se sigue calculando y almacenando, de modo que la trazabilidad del desglose no se pierde.

CASO 5 · MISMA ENTRADA	SCORE	TIER BORRADOR
Antes del ajuste	56	B Confirmaba el free para las ocho personas
Después del ajuste	38	C No confirma. Pide reducir a tres personas y adjuntar prueba de los grupos de difusión

La regla se incorporó simultáneamente al *system prompt* y a la base de conocimiento. Duplicarla no es redundancia por descuido: la documentación sostiene que la base de conocimiento es la fuente única de los criterios de evaluación, y un criterio que viviera sólo dentro del prompt volvería falsa esa afirmación.

Repositorio y enlaces

Todo el material de la entrega está publicado en un repositorio público: github.com/tomasetiennot-afk/ecosistema-frees-ia

ARCHIVO	DESCRIPCIÓN
README.md	Índice del proyecto, enlaces públicos y guía de lectura
documentacion-entrega-final.pdf	Este documento. Cubre los cinco criterios de evaluación
blueprints/intake-y-scoring.json	Lógica completa del escenario 1 exportada desde el orquestador, con el router y el error handler
blueprints/notificacion-post-aprobacion.json	Lógica completa del escenario 2, con el filtro anti-bucle
evidencias/	Capturas de las ejecuciones, las rutas de contingencia, el HITL y el panel de control

Enlaces del ecosistema

RECURSO	DIRECCIÓN
Repositorio	github.com/tomasetiennot-afk/ecosistema-frees-ia
Formulario público	tally.so/r/1Aeqyb
Vista de solicitudes	airtable.com/appIVK7AvA6qOrnQA/shrxQ5B6QKmlZdJVY
Vista de errores	airtable.com/appIVK7AvA6qOrnQA/shr3Exf9iuTwaZ3S9

CIERRE

Del operador al arquitecto

El proceso que este sistema reemplaza consistía en leer mensajes sueltos, decidir de memoria y no anotar nada. Lo que queda en su lugar no es sólo más rápido: es auditable. Cada solicitud tiene un puntaje con su composición, cada decisión tiene una justificación escrita, cada fallo queda registrado con el módulo donde ocurrió, y los criterios que gobiernan las evaluaciones viven en una tabla que se edita sin tocar el flujo.

La decisión de diseño más relevante no fue técnica. Fue advertir que el proceso original no se podía automatizar tal como estaba: un pedido en texto libre por mensaje directo no tiene estructura, no deja constancia y no admite consentimiento explícito. Automatizar ese circuito habría producido desorden más rápido. Rediseñar la entrada antes de conectar herramientas es lo que hizo posible todo lo demás.

La segunda decisión relevante fue dónde poner el freno. El sistema es capaz de redactar y enviar la respuesta sin ayuda, y deliberadamente no lo hace. Entre una evaluación correcta y un mensaje enviado a una persona real hay una diferencia que ninguna métrica de exactitud captura, y esa diferencia es exactamente el lugar donde tiene que estar el criterio humano.